

BLM4522 — AĞ TABANLI PARALEL DAĞITIM SİSTEMLERİ

Proje 2: Veritabanı Yedekleme ve Felaketten Kurtarma Planı

Proje 6: Veritabanı Yükseltme ve Sürüm Yönetimi

Ad Soyad: Selim Bedirhan Öztürk

Öğrenci No: 20291277

PROJE 2: VERİTABANI YEDEKLEME VE FELAKETTEN KURTARMA PLANI

1. Giriş ve Amaç

Bu projenin amacı, bir veritabanı yönetim sistemi üzerinde kapsamlı bir yedekleme ve felaketten kurtarma planı tasarlamak ve uygulamaktır. Veritabanı yedekleme, modern bilgi sistemlerinin en kritik bileşenlerinden biridir. Donanım arızaları, yazılım hataları, insan kaynaklı hatalar veya siber saldırılar gibi beklenmedik durumlarla karşılaşıldığında verilerin kurtarılabilmesi büyük önem taşır.

Proje kapsamında PostgreSQL 14 üzerinde **tam (full) yedekleme**, **fark (differential) yedekleme**, **artık (incremental/WAL) yedekleme**, **zamanlanmış otomatik yedekleme**, **felaketten kurtarma senaryoları**, **Point-in-Time Recovery (PITR)** ve **yedek doğrulama testleri** ele alınmıştır.

2. Kullanılan Teknolojiler

Teknoloji	Açıklama
PostgreSQL 14	Veritabanı yönetim sistemi
pg_dump / pg_restore	Mantıksal yedekleme ve geri yükleme
pg_basebackup	Fiziksel yedekleme (PITR için)
WAL Arşivleme	Transaction log tabanlı sürekli yedekleme
Bash Scripting	Otomasyon ve raporlama
Cron	Zamanlanmış görev yönetimi
gzip	Yedek dosyalarının sıkıştırılması

3. Veritabanı Tasarımı — E-Ticaret Sistemi

Projede gerçekçi bir senaryo sunmak üzere bir **E-Ticaret veritabanı** tasarlanmıştır. Toplam 8 ana tablo, view'lar, trigger'lar ve 15.000'den fazla kayıt ile yedekleme testleri yapılmıştır.

Tablo	Açıklama	Kayıt
kategoriler	Ürün kategorileri (hiyerarşik)	20
tedarikciler	Tedarikçi firma bilgileri	15
musteriler	Müşteri bilgileri	500
urunler	Ürün kataloğu	200
siparisler	Sipariş kayıtları	3.000
siparis_detaylari	Sipariş kalemleri	8.000
odemeler	Ödeme işlemleri	2.500
stok_hareketleri	Stok giriş/çıkış hareketleri	1.200+
TOPLAM		15.000+

Veritabanı ayrıca 3 View (`v_siparis_ozeti` , `v_stok_durumu` , `v_gunluk_satis`), 2 Trigger/Fonksiyon (`fn_stok_guncelle` , `fn_siparis_toplam_guncelle`) ve 15+ indeks içermektedir.

4. Yedekleme Stratejileri

4.1 Tam (Full) Yedekleme

Dört farklı format ile tam yedekleme uygulanmıştır:

Format	Komut	Avantaj	Kullanım
Custom (.dump)	<code>pg_dump -Fc -Z 6</code>	Sıkıştırma, seçici restore	Günlük yedek
Plain SQL	<code>pg_dump -Fp --create</code>	Okunabilir, sürüm uyumu	Denetim
Compressed	<code>pg_dump gzip -9</code>	Alan tasarrufu	Arşiv
Tar (.tar)	<code>pg_dump -Ft</code>	Dosya sistemi yapısı	Taşınabilirlik

4.2 Fark (Differential) Yedekleme

- **Şema (DDL) yedekleme:** `pg_dump --schema-only` ile sadece tablo yapıları
- **Sadece veri yedekleme:** `pg_dump --data-only --column-inserts`
- **Tablo bazlı kısmi yedekleme:** `pg_dump --table=<tablo>`
- **Tarih filtreli yedekleme:** `\COPY` ile son N günde değişen veriler

4.3 Artık (Incremental) Yedekleme — WAL

PostgreSQL'in Write-Ahead Logging (WAL) altyapısı kullanılmıştır. WAL konfigürasyonu (`wal_level = replica` , `archive_mode = on`), `pg_basebackup` ile baz yedekleme ve WAL arşivleme simülasyonu uygulanmıştır.

5. Zamanlanmış Otomatik Yedekleme

Zamanlama	İşlem	Script
Her gün 02:00	Günlük otomatik yedekleme	<code>auto_backup.sh</code>
Her Pazar 03:00	Haftalık tam yedekleme	<code>03_tam_yedekleme.sh</code>
Hafta içi saatlik	Fark yedekleme	<code>04_fark_yedekleme.sh</code>

Saklama Politikası: Günlük yedekler 7 gün, eski loglar 30 gün sonra otomatik temizlenir.

6. Felaketten Kurtarma Senaryoları

Dört farklı felaket senaryosu simüle edilmiş ve kurtarma işlemleri başarıyla gerçekleştirilmiştir:

Senaryo 1 — Yanlışlıkla Tablo Silme: `DROP TABLE stok_hareketleri CASCADE` sonrası `pg_restore --table=stok_hareketleri` ile sadece ilgili tablonun geri yüklenmesi.

Senaryo 2 — Veritabanı Tam Bozulma: Yeni bir test veritabanına `pg_restore --clean --if-exists --single-transaction` ile tam restore. Kayıt sayıları karşılaştırılarak doğrulanmıştır.

Senaryo 3 — Yanlış Veri Güncelleme: `UPDATE urunler SET birim_fiyat = 0` ile tüm fiyatların sıfırlanması, ardından yedekten doğru fiyatların geri yüklenmesi.

Senaryo 4 — Kitleseel Veri Silme: İstanbul müşterilerinin bağımlı verileriyle birlikte silinmesi, sonra tam restore ile geri yüklenmesi.

7. Yedek Doğrulama ve Test Sistemi

Test Grubu	İçerik
Yedek Oluşturma	Custom ve Plain SQL format yedek oluşturma
Bütünlük Kontrolleri	pg_restore --list, dosya boyutu, SQL syntax
Geri Yükleme	Test DB'ye restore ve kayıt sayısı eşleşmesi
Veri Bütünlüğü	FK ilişkileri, indeksler, view'lar, fonksiyonlar
Performans	Yedekleme süresi ölçümü, boyut karşılaştırması

8. Proje 2 Sonuç

- PostgreSQL yedekleme araçlarının (pg_dump, pg_restore, pg_basebackup) etkin kullanımı gerçekleştirildi
- Dört farklı yedekleme formatının avantaj/dezavantajları karşılaştırıldı
- WAL tabanlı sürekli yedekleme altyapısı uygulandı
- Gerçekçi felaket senaryolarında başarılı veri kurtarma gösterildi
- Cron ile zamanlanmış otomasyon ve yedek doğrulama testleri yapıldı

PROJE 6: VERİTABANI YÜKSELTME VE SÜRÜM YÖNETİMİ

1. Giriş ve Amaç

Bu proje, veritabanı sürüm yükseltme planlaması ve uygulamasını, uyumluluk testlerini, geçiş stratejilerini ve geri alma (rollback) mekanizmalarını kapsamlı bir şekilde göstermektedir. Bir **Kütüphane Yönetim Sistemi** üzerinde 3 aşamalı sürüm geçişi (v1.0 → v2.0 → v3.0) uygulanmıştır.

Anahtar Kavramlar:

- Schema Migration:** Veritabanı şemasının kontrollü evrimi
- Version Tracking:** Sürüm geçmişinin veritabanında saklanması
- Pre/Post Upgrade Checks:** Yükseltme öncesi ve sonrası kontroller
- Rollback:** Başarısız geçiş durumunda geri alma
- Atomik Migration:** Transaction blokları içinde güvenli geçiş

2. Kullanılan Teknolojiler

Teknoloji	Açıklama
PostgreSQL 14	Veritabanı yönetim sistemi
psql	İnteraktif SQL istemcisi
Bash Scripting	Otomasyon ve test scriptleri
pg_dump	Yükseltme öncesi yedekleme
Transaction (BEGIN/COMMIT)	Atomik migration işlemleri

3. Sürüm Tasarımı ve Evrim

3.1 v1.0 — Temel Kütüphane Sistemi

Tablo	Açıklama	Kayıt
schema_version	Sürüm takip tablosu	1
yazarlar	Yazar bilgileri	40
kategoriler	Kitap kategorileri	12
kitaplar	Kitap kataloğu	200
uyeler	Üye kayıtları	400
odunc_islemleri	Ödünç alma işlemleri	2.000
TOPLAM		~2.650

7 indeks: ISBN, yazar, kategori, TC kimlik, ödünç işlemleri üzerine.

3.2 v2.0 — Genişletilmiş Sistem

Yeni Tablolar:

- yayinevleri** — Yayınevi bilgileri (8 kayıt)
- kitap_yazarlar** — Çoka-çok yazar-kitap ilişkisi (200 kayıt)
- cezalar** — Gecikme ceza sistemi (~200 kayıt)
- rezervasyonlar** — Kitap rezervasyonu (300 kayıt)

Kolon Değişiklikleri:

- kitaplar: **+yayinevi_id**, **+dil**, **+aciklama**, **+etiketler**
- uyeler: **+uyelik_tipi**, **+dogum_tarihi**, **+max_odunc**
- yazarlar: **+biyografi**
- odunc_islemleri: **+notlar**, **+uzatma_sayisi**

Yeni Objeler: 2 View (**v_kitap_detay**, **v_uye_ceza_durumu**), 1 Fonksiyon (**fn_ceza_hesapla**), 7 İndeks

3.3 v3.0 — Tam Özellikli Sistem

Yeni Tablolar:

- dijital_kitaplar** — E-kitap/sesli kitap (80 kayıt)
- etkinlikler** — Kütüphane etkinlikleri (20 kayıt)
- etkinlik_katilim** — Etkinlik katılım kayıtları
- degerlendirmeler** — Kitap puanlama/yorum (~400 kayıt)
- bildirimler** — Üye bildirimleri (600 kayıt)

Kolon Değişiklikleri:

- kitaplar: **+ort_puan**, **+degerlendirme_sayisi**, **+dijital_mevcut**

- uyeler: **+toplam_odunc** , **+gecikme_sayisi** , **+puan**

Yeni Objeler: 2 View, 1 Trigger (**trg_puan_guncelle**), 6 İndeks

4. Schema Migration Sistemi

4.1 schema_version Tablosu

Her migration işleminin detaylı kaydı tutulur:

Kolon	Tip	Açıklama
version_id	SERIAL	Benzersiz kimlik
version_no	VARCHAR	Sürüm numarası (1.0.0, 2.0.0, 3.0.0)
aciklama	TEXT	Değişiklik özeti
migration_dosya	VARCHAR	Migration SQL dosyası
uygulama_tarihi	TIMESTAMP	Uygulama zamanı
sure_ms	INTEGER	İşlem süresi (ms)
durum	VARCHAR	basarili / basarisiz / geri_alindi
geri_alma_dosya	VARCHAR	Rollback dosya adı

4.2 Migration Dosya Yapısı

Her migration 6 adımı izler:

1. **Sürüm kontrolü** — Mevcut sürümün beklenenle eşleştiğini doğrula
2. **Transaction başlat** — **BEGIN** ile atomik işlem
3. **Değişiklikleri uygula** — CREATE TABLE, ALTER TABLE, CREATE INDEX
4. **Veri migrasyonu** — Mevcut verilerin yeni yapıya uyarlanması
5. **Sürüm kaydını güncelle** — schema_version'a yeni kayıt
6. **Transaction bitir** — **COMMIT**

5. Yükseltme Öncesi Kontroller

#	Kontrol	Açıklama
1	Bağlantı	PostgreSQL ve DB erişimi
2	Sürüm bilgisi	Mevcut sürüm ve geçmiş
3	DB durumu	Tablo, indeks, view, fonksiyon sayıları
4	Veri bütünlüğü	FK ihlali, NULL, duplicate kontrol
5	Aktif işlemler	Uzun çalışan sorgu kontrolü
6	Disk alanı	Yeterli alan kontrolü
7	Yedek	Yükseltme öncesi otomatik yedek

6. Migration Yönetimi ve Otomasyon

Migration yönetici scripti mevcut sürümü otomatik tespit ederek sıradaki migration'ı uygular:

- Mevcut sürüm v1.0 ise → v1→v2 çalıştırır, sonra v2→v3
- Mevcut sürüm v2.0 ise → sadece v2→v3 çalıştırır
- Mevcut sürüm v3.0 ise → zaten güncel

Her migration **BEGIN...COMMIT** transaction bloğu içinde çalışır. Herhangi bir hata durumunda tüm değişiklikler otomatik geri alınır (PostgreSQL DDL transaction desteği).

7. Rollback Stratejisi

Her migration dosyası için karşılık gelen rollback dosyası bulunur:

Migration	Rollback
v1_to_v2_migration.sql	v2_to_v1_rollback.sql
v2_to_v3_migration.sql	v3_to_v2_rollback.sql

Rollback Süreci:

1. Yeni eklenen tablolar **DROP TABLE CASCADE** ile silinir
2. Eklenen kolonlar **ALTER TABLE DROP COLUMN** ile kaldırılır
3. Yeni view ve fonksiyonlar silinir
4. schema_version tablosunda durum **geri_alindi** olarak güncellenir
5. Rollback kaydı eklenir

Demo Senaryosu:

v1.0 kur → v2.0 yüksel → v3.0 yüksel → v3.0 geri al (v2.0'a dön) → v3.0 i

8. Yükseltme Sonrası Uyumluluk Testleri

5 test grubunda toplam 28 test uygulanmıştır:

Test Grubu	Adet	Sonuç
Tablo varlık kontrolü (v1+v2+v3)	15	15/15 GEÇTİ
Kolon kontrolü	7	7/7 GEÇTİ
Veri bütünlüğü	3	3/3 GEÇTİ
View / Fonksiyon	3	3/3 GEÇTİ
Performans	1	1/1 GEÇTİ
TOPLAM	28	28/28 BAŞARILI

9. Sürüm Karşılaştırması

Özellik	v1.0	v2.0	v3.0
Tablo	6	10	15
Kolon	~35	~55	~70
İndeks	7	14	20
View	0	2	4
Fonksiyon	0	1	2
Trigger	0	0	1
FK	5	12	17
Toplam Kayıt	~2.650	~3.350	~4.460

10. Proje 6 Sonuç

- Schema migration kavramı PostgreSQL üzerinde transaction-tabanlı olarak uygulandı
- Otomatik sürüm tespiti ve zincirli migration mekanizması kuruldu
- Kapsamlı pre/post upgrade kontrolleri (28 test) başarıyla geçildi
- Çalışan rollback mekanizması ile güvenli geri alma doğrulandı
- Veri bütünlüğü korunarak 3 aşamalı sürüm yükseltme başarıyla tamamlandı

Kaynakça

1. PostgreSQL Documentation — Backup and Restore

<https://www.postgresql.org/docs/14/backup.html>

2. PostgreSQL Documentation — Continuous Archiving and PITR
<https://www.postgresql.org/docs/14/continuous-archiving.html>
3. PostgreSQL Documentation — pg_dump
<https://www.postgresql.org/docs/14/app-pgdump.html>
4. PostgreSQL Documentation — pg_restore
<https://www.postgresql.org/docs/14/app-pgrestore.html>
5. PostgreSQL Documentation — ALTER TABLE
<https://www.postgresql.org/docs/14/sql-altertable.html>
6. PostgreSQL Wiki — Transactional DDL
https://wiki.postgresql.org/wiki/Transactional_DDL_in_PostgreSQL